L02 — MapReduce với Hadoop Streaming (Python)

Cluster: dùng lại cluster L01 (1 NN, 3 DN, RM, NM, JobHistory). Log gốc trong các file `0*.txt` và `*combiner.log`; ảnh ở `img/`.

File nộp

File	Nội dung
`wc_mapper.py`, `wc_reducer.py`	TODO 1–3 (word count)
`zones_mapper.py`	TODO 4–6 (map-side join với `taxi_zone_lookup.csv`, counter `BAD_RECORDS`)
`stretch_avg_fare/`	Stretch: giá cước trung bình theo zone với combiner đúng `(sum, count)`

wc_mapper.py

```
#!/usr/bin/env python3
"""Word count -- MAPPER (Hadoop Streaming).

Hadoop feeds the input split to this script line by line on STDIN.
Whatever you print to STDOUT is the map output: one  key<TAB>value  per line.

Goal: for every word in the line, emit  "<word>\t1".
Normalise words to lower case and keep only letters and apostrophes,
e.g. "Hadoop's HDFS, hadoop!" -> hadoop's 1 / hdfs 1 / hadoop 1
"""
import re
import sys

WORD = re.compile(r"[a-z][a-z']*")

for line in sys.stdin:
    for word in WORD.findall(line.lower()):
        print(f"{word}\t1")
```

wc_reducer.py

```
#!/usr/bin/env python3
"""Word count -- REDUCER (Hadoop Streaming).

Hadoop SORTS the map output by key before calling the reducer, so all lines
with the same word arrive one after another:
    apache\t1
    apache\t1
    hadoop\t1
Unlike the Java API you do NOT get (key, [values]) -- you must detect the
key change yourself.

Goal: print "<word>\t<total>" once per word.
"""
import sys

current_word, current_count = None, 0

for line in sys.stdin:
    word, _, count = line.rstrip("\n").partition("\t")
    count = int(count)
    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word, current_count = word, count

if current_word is not None:  # the last word's total
    print(f"{current_word}\t{current_count}")
```

zones_mapper.py

```
#!/usr/bin/env python3
"""Trips per pickup zone -- MAPPER (Hadoop Streaming).

Input lines are headerless taxi CSV rows (labs/datasets/parquet_to_csv.py):
  0 VendorID, 1 tpep_pickup_datetime, 2 tpep_dropoff_datetime, 3 passenger_count,
  4 trip_distance, 5 RatecodeID, 6 store_and_fwd_flag, 7 PULocationID,
  8 DOLocationID, 9 payment_type, 10 fare_amount, ... 16 total_amount, ...

Goal: emit "<zone>\t1" per trip.  If the file taxi_zone_lookup.csv is present in
the task's working directory (ship it with  -files .../taxi_zone_lookup.csv),
translate the numeric PULocationID into "Borough/Zone"  -> a MAP-SIDE JOIN.
"""
import csv
import os
import sys

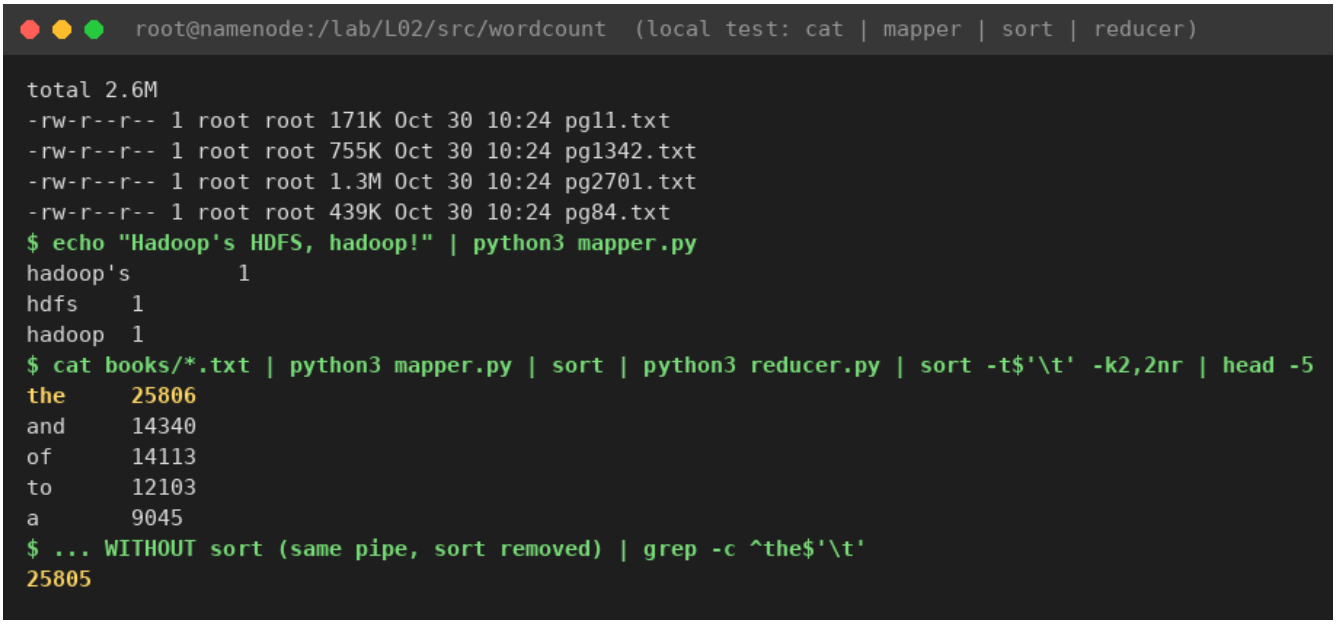
PU_COL = 7

zones = {}
if os.path.exists("taxi_zone_lookup.csv"):
    with open("taxi_zone_lookup.csv", newline="") as f:
        for row in csv.DictReader(f):  # header: LocationID,Borough,Zone,service_zone
            zones[row["LocationID"]] = f"{row['Borough']}/{row['Zone']}"

for line in sys.stdin:
    fields = line.rstrip("\n").split(",")
    if len(fields) <= PU_COL or not fields[PU_COL].strip().isdigit():
        sys.stderr.write("reporter:counter:TaxiLab,BAD_RECORDS,1\n")
        continue
    pu = fields[PU_COL].strip()
    print(f"{zones.get(pu, pu)}\t1")
```

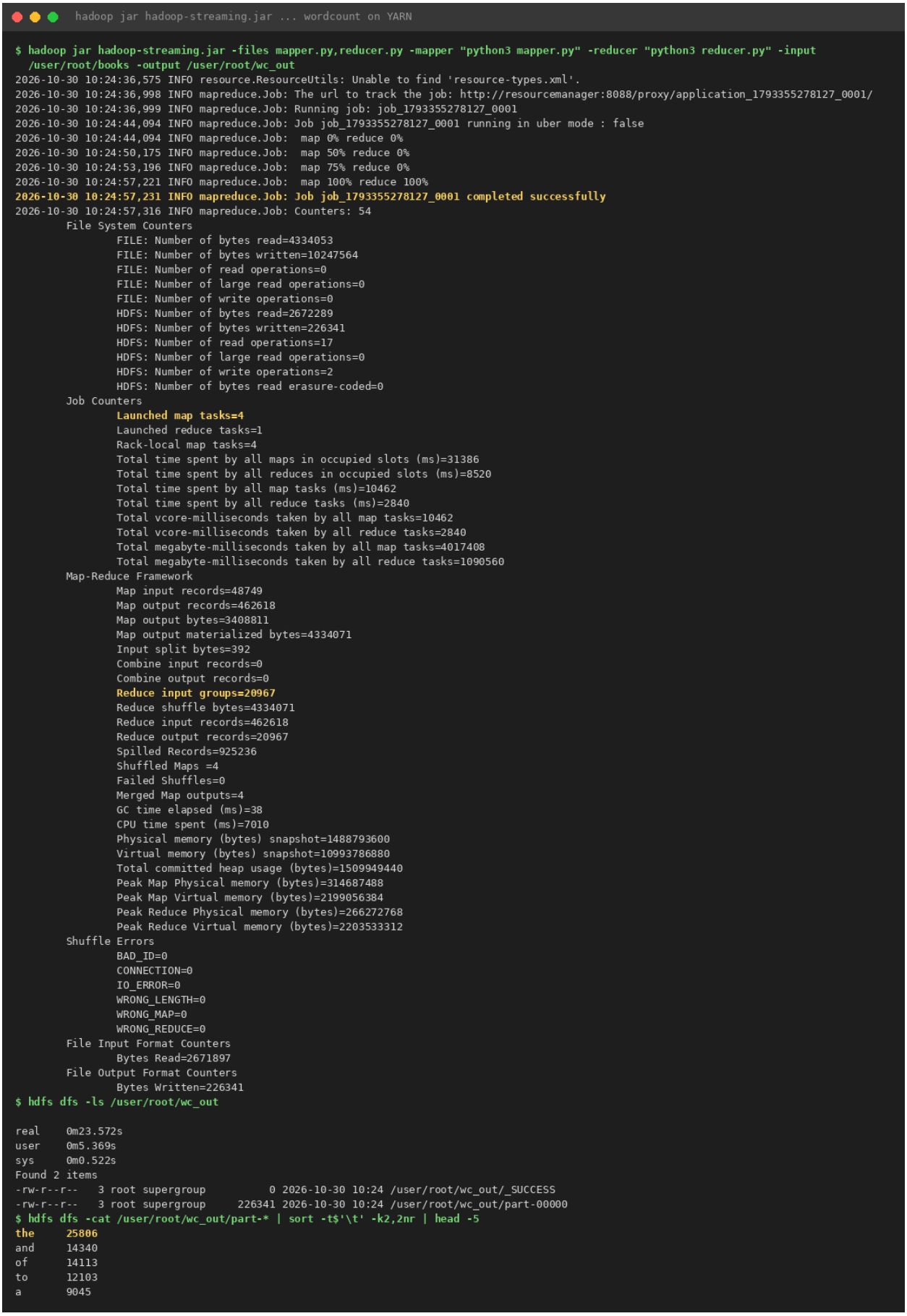
1-2. Word count: test local và chạy trên YARN

Test local (`cat | mapper | sort | reducer`) cho top 5 `the 25806, and 14340, of 14113, to 12103, a 9045`. Khi **bỏ `sort`**, reducer in `the` ra **25,805 dòng** (mỗi lần xuất hiện gần như là một "nhóm" riêng) thay vì 1 dòng tổng.



Hình 1: local pipe

Trên YARN: `Launched map tasks=4` (mỗi cuốn sách < 1 block là 1 split), `Reduce input groups=20967`; top 5 giống hệt bản local.



Hình 2: wordcount yarn

3–4. Dữ liệu taxi CSV và map-side join

1,000,000 dòng (92.8 MB) đưa lên HDFS với block 16 MB → **6 block → 6 map task**. Test local trên 1,000 dòng: `Manhattan/East Village 59`, `Manhattan/Lincoln Square East 55`. Dòng hỏng được đếm qua stderr `reporter:counter:TaxiLab,BAD_RECORDS,1` (dữ liệu thật có 0 dòng hỏng nên counter không xuất hiện trong job).

```
parquet_to_csv (1M rows) -> HDFS 16 MB blocks; zones mapper local test

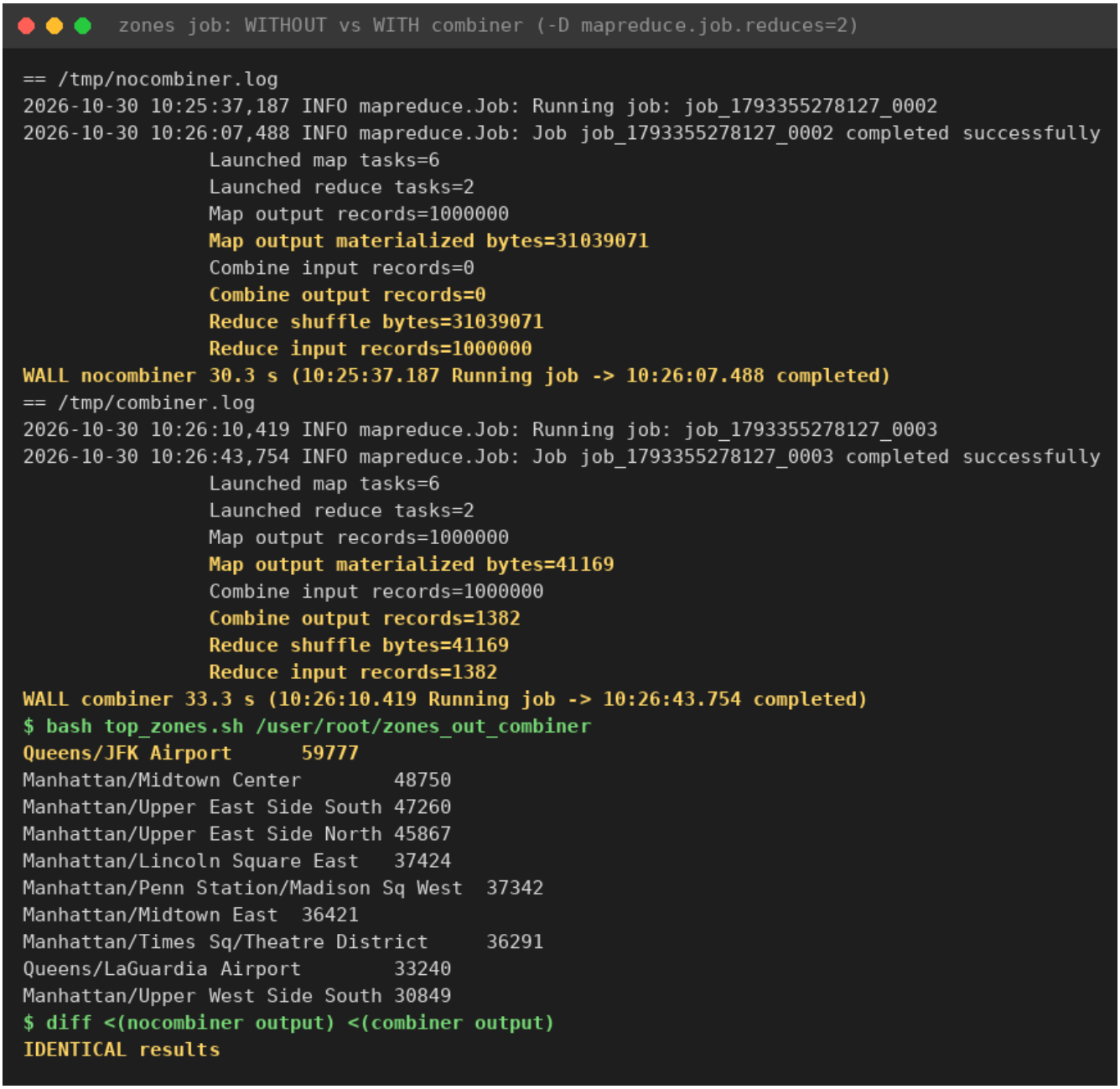
wrote 1,000,000 rows -> /lab/L02/work/yellow_2024-01_1M.csv (92.8 MB)
$ head -2 yellow_2024-01_1M.csv
2,2024-01-01 00:57:55,2024-01-01 01:17:43,1,1.72,1,N,186,79,2,17.7,1,0.5,0,0,1,22.7,2.5,0
1,2024-01-01 00:03:00,2024-01-01 00:09:36,1,1.8,1,N,140,236,1,10,3.5,0.5,3.75,0,1,18.75,2.5,0
$ hdfs fsck /user/root/taxi_csv -files -blocks | grep "block(s)"
/user/root/taxi_csv/yellow_2024-01_1M.csv 92847108 bytes, replicated: replication=3, 6 block(s):  OK
$ head -1000 yellow_2024-01_1M.csv | python3 mapper.py | sort | python3 reducer.py | sort -t$'\t' -k2,2nr | head -3
Manhattan/East Village  59
Manhattan/Lincoln Square East   55
Manhattan/Upper West Side South 45
$ printf "garbage\nx,y,z,a,b,c,d,NOTNUM,1\n" | python3 mapper.py   # bad records -> stderr counter
reporter:counter:TaxiLab,BAD_RECORDS,1
reporter:counter:TaxiLab,BAD_RECORDS,1
```

Hình 3: taxi csv

5. Combiner và không combiner

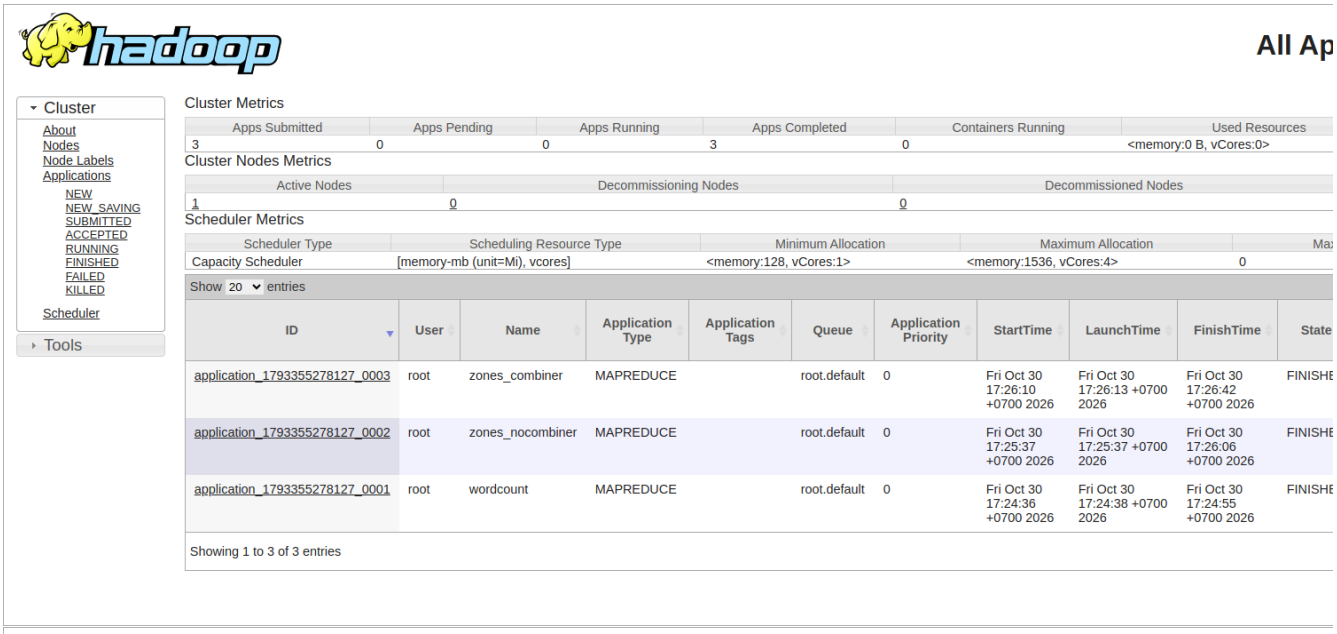
Counter	Không combiner	Có combiner
Launched map tasks	6	6
Map output records	1,000,000	1,000,000
Combine output records	0	1,382
Map output materialized bytes	31,039,071	41,169
Reduce shuffle bytes	31,039,071	41,169
Reduce input records	1,000,000	1,382
Wall-clock time	30.3 s	33.3 s

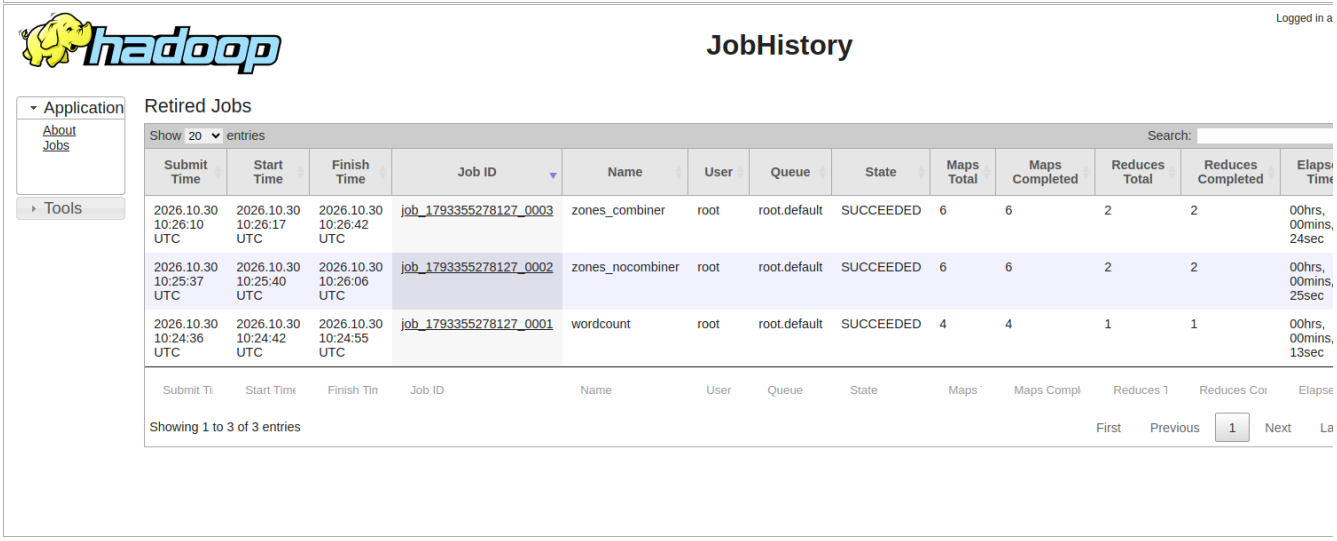
Kết quả hai lần chạy **giống hệt** (`diff` rỗng). Top zone: `Queens/JFK Airport 59777`, `Manhattan/Midtown Center 48750`.

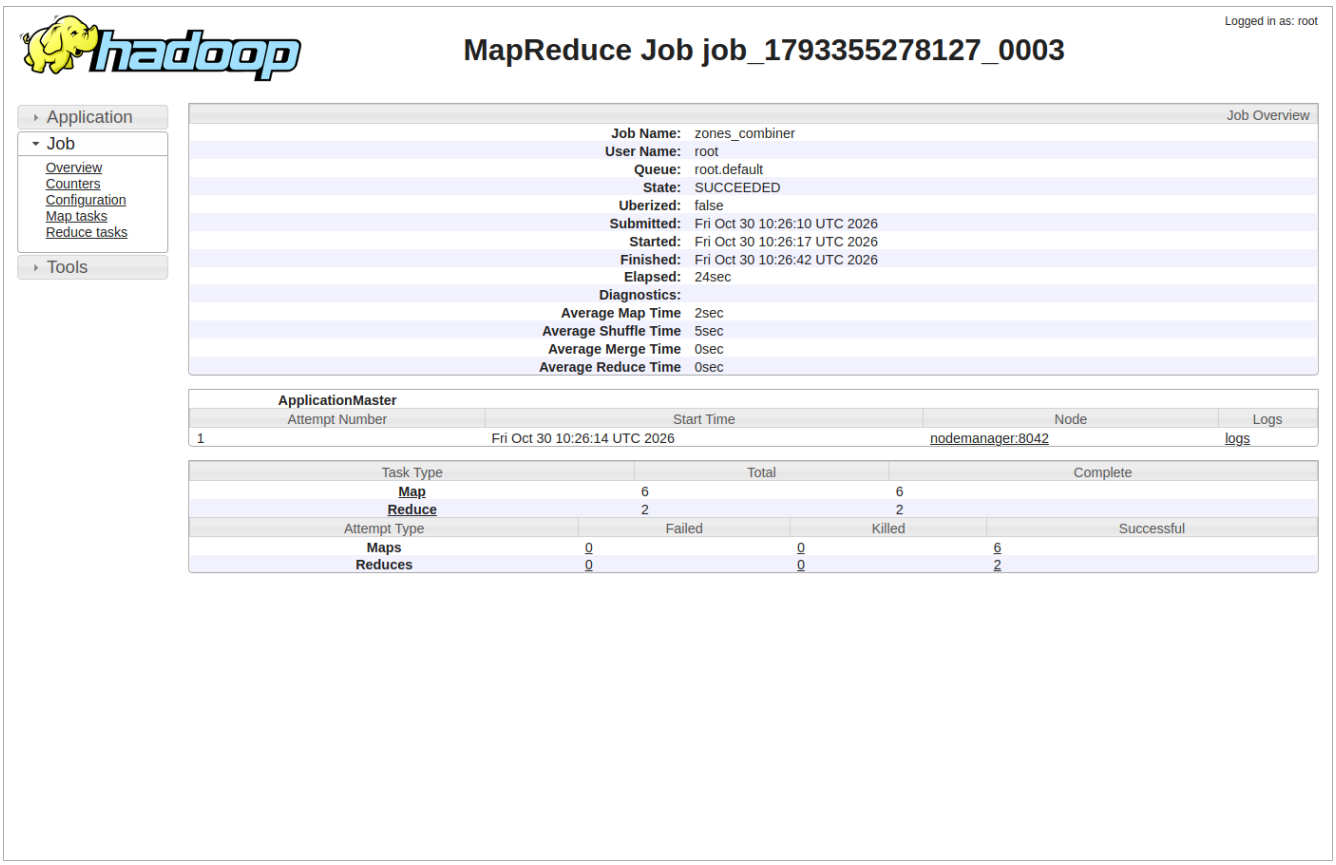


Hình 4: combiner table

6. YARN và JobHistory UI






Logged in as: root
MapReduce Job job_1793355278127_0003
Application
Job
Overview
Counters
Configuration
Map tasks
Reduce tasks
Tools
Job Overview
Job Name: zones_combiner
User Name: root
Queue: root.default
State: SUCCEEDED
Uberized: false
Submitted: Fri Oct 30 10:26:10 UTC 2026
Started: Fri Oct 30 10:26:17 UTC 2026
Finished: Fri Oct 30 10:26:42 UTC 2026
Elapsed: 24sec
Diagnostics:
Average Map Time 2sec
Average Shuffle Time 5sec
Average Merge Time 0sec
Average Reduce Time 0sec
ApplicationMaster
Attempt Number Start Time Node Logs
1 Fri Oct 30 10:26:14 UTC 2026 nodemanager:8042 logs
Task Type Total Complete
Map 6 6
Reduce 2 2
Attempt Type Failed Killed Successful
Maps 0 0 6
Reduces 0 0 2

Logged in as: root

Counters for job_1793355278127_0002

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Counters			
	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	31,039,011	31,039,011
	FILE: Number of bytes written	32,936,475	31,671,321	64,607,796
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
	Name	Map	Reduce	Total
Job Counters	Killed reduce tasks	0	0	1
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,638,208
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,996,992
	Total time spent by all map tasks (ms)	0	0	17,287
	Total time spent by all maps in occupied slots (ms)	0	0	51,861
	Total time spent by all reduce tasks (ms)	0	0	13,013
	Total time spent by all reduces in occupied slots (ms)	0	0	39,039
	Total vcore-milliseconds taken by all map tasks	0	0	17,287
	Total vcore-milliseconds taken by all reduce tasks	0	0	13,013
	Name	Map	Reduce	Total
Map-Reduce Framework	Combine input records	0	0	0
	Combine output records	0	0	0
	CPU time spent (ms)	10,500	4,550	15,050
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	74	57	131
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	31,039,071	0	31,039,071
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	332,361,728	0	332,361,728
	Peak Map Virtual memory (bytes)	2,204,459,008	0	2,204,459,008
	Peak Reduce Physical memory (bytes)	0	318,996,480	318,996,480
	Peak Reduce Virtual memory (bytes)	0	2,203,987,968	2,203,987,968
	Physical memory (bytes) snapshot	1,953,869,824	590,958,592	2,544,828,416
	Reduce input groups	0	247	247
	Reduce input records	0	1,000,000	1,000,000
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	31,039,071	31,039,071
	Shuffled Maps	0	12	12
	Spilled Records	1,000,000	1,000,000	2,000,000
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104

Logged in as: root

Counters for job_1793355278127_0003

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Counters			
	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	41,109	41,109
	FILE: Number of bytes written	1,940,751	674,145	2,614,896
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
Job Counters	Name	Map	Reduce	Total
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,680,832
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,359,552
	Total time spent by all map tasks (ms)	0	0	17,398
	Total time spent by all maps in occupied slots (ms)	0	0	52,194
	Total time spent by all reduce tasks (ms)	0	0	11,353
	Total time spent by all reduces in occupied slots (ms)	0	0	34,059
	Total vcore-milliseconds taken by all map tasks	0	0	17,398
	Total vcore-milliseconds taken by all reduce tasks	0	0	11,353
Map-Reduce Framework	Name	Map	Reduce	Total
	Combine input records	1,000,000	0	1,000,000
	Combine output records	1,382	0	1,382
	CPU time spent (ms)	12,380	1,640	14,020
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	72	26	98
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	41,169	0	41,169
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	328,896,512	0	328,896,512
	Peak Map Virtual memory (bytes)	2,201,120,768	0	2,201,120,768
	Peak Reduce Physical memory (bytes)	0	251,559,936	251,559,936
	Peak Reduce Virtual memory (bytes)	0	2,205,458,432	2,205,458,432
	Physical memory (bytes) snapshot	1,954,152,448	501,567,488	2,455,719,936
	Reduce input groups	0	247	247
	Reduce input records	0	1,382	1,382
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	41,169	41,169
	Shuffled Maps	0	12	12
	Spilled Records	1,382	1,382	2,764
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104
	Virtual memory (bytes) snapshot	13,186,910,208	4,407,558,144	17,594,477,440

```
yarn application -list / mapred job -list all

$ yarn application -list -appStates FINISHED
Total number of applications (application-types: [], states: [FINISHED] and tags: []):3
                Application-Id      Application-Name        Application-Type          User           Queue                   State             Final-State             Progress
  Tracking-URL
application_1793355278127_0003        zones_combiner               MAPREDUCE          root      root.default              FINISHED               SUCCEEDED                100%
  http://historyserver:19888/jobhistory/job/job_1793355278127_0003
application_1793355278127_0001             wordcount               MAPREDUCE          root      root.default              FINISHED               SUCCEEDED                100%
  http://historyserver:19888/jobhistory/job/job_1793355278127_0001
application_1793355278127_0002      zones_nocombiner               MAPREDUCE          root      root.default              FINISHED               SUCCEEDED                100%
  http://historyserver:19888/jobhistory/job/job_1793355278127_0002
$ mapred job -list all
Total jobs:3
                  JobId                                JobName            State           StartTime      UserName           Queue      Priority      UsedContainers  RsvdC
 job_1793355278127_0003                        zones_combiner         SUCCEEDED       1793355970353          root    root.default       DEFAULT                 N/A
 job_1793355278127_0001                             wordcount         SUCCEEDED       1793355876697          root    root.default       DEFAULT                 N/A
 job_1793355278127_0002                       zones_nocombiner         SUCCEEDED       1793355937120          root    root.default       DEFAULT                 N/A
```

Stretch (bonus): giá cước trung bình theo zone với combiner đúng

Mapper emit `zone \t fare,1`; combiner cộng dồn `(sum, count)` và xuất **cùng định dạng** với input; reducer cộng tiếp rồi mới chia `sum/count`. Kết quả JFK `59.71` trên `59,777` chuyến, trùng với bản chạy local không combiner, nghĩa là combiner không làm sai kết quả. Shuffle chỉ còn 56,289 byte.

```
stretch: average fare per zone with a correct (sum,count) combiner

$ hadoop jar streaming -mapper mapper.py -combiner combiner.py -reducer reducer.py -input /user/root/taxi_csv -output /user/root/avg_fare_out
2026-10-30 10:28:00,220 INFO mapreduce.Job: Job job_1793355278127_0004 completed successfully
                Combine input records=1000000
                Combine output records=1382
                Reduce shuffle bytes=56289
$ top 10 zones by trips: zone / avg_fare / trips
Queens/JFK Airport      59.71   59777
Manhattan/Midtown Center        15.33   48750
Manhattan/Upper East Side South 11.91   47260
Manhattan/Upper East Side North 12.49   45867
Manhattan/Lincoln Square East   13.30   37424
Manhattan/Penn Station/Madison Sq West  15.72   37342
Manhattan/Midtown East  14.71   36421
Manhattan/Times Sq/Theatre District     18.20   36291
Queens/LaGuardia Airport        40.87   33240
Manhattan/Upper West Side South 13.36   30849
$ check vs local (no combiner) for JFK:
Queens/JFK Airport      59.71   59777
```

stretch_avg_fare/combiner.py

```
#!/usr/bin/env python3
"""Average fare -- COMBINER: adds up (sum, count) pairs per key. Output has the same shape as its
input, so it may run 0, 1 or many times without changing the final result."""
import sys

current_key, total, count = None, 0.0, 0

for line in sys.stdin:
    key, _, value = line.rstrip("\n").partition("\t")
    s, _, c = value.partition(",")
    if key == current_key:
        total, count = total + float(s), count + int(c)
    else:
        if current_key is not None:
            print(f"{current_key}\t{total},{count}")
        current_key, total, count = key, float(s), int(c)

if current_key is not None:
    print(f"{current_key}\t{total},{count}")
```

Trả lời checkpoint

Q1. Lệnh `sort` đóng vai trò **shuffle & sort**: nó gom các key giống nhau nằm liền nhau. Thiếu nó, reducer thấy cùng một từ ở nhiều đoạn không liền nhau nên in ra nhiều lần với số đếm cục bộ (thực nghiệm: `the` bị in 25,805 lần).

Q2. Streaming chỉ đưa cho reducer một **luồng dòng đã sort**; khái niệm "nhóm" `(key, Iterable<values>)` nằm trong framework Java. Reducer Python phải tự phát hiện ranh giới key (`current_key`) và nhớ in key cuối cùng.

Q3. 6 map task = 6 input split = 6 block HDFS 16 MB của file CSV 92.8 MB (mặc định mỗi block là một split). Wordcount có 4 map vì mỗi cuốn sách nhỏ hơn 1 block nên là 1 split. **2 reduce task** vì đặt `-D mapreduce.job.reduces=2` (mặc định là 1): số reducer do người dùng chọn, không suy ra từ dữ liệu.

Q4. Shuffle giảm **~754 lần** (31,039,071 → 41,169 byte). Mỗi map task chỉ xuất một bản ghi cho mỗi **zone khác nhau mà nó gặp** (~230–260 zone) × 6 map ≈ 1,382. Chỉ các tổng cục bộ này đi qua mạng.

Q5. Combiner không nhanh hơn (33.3 s so với 30.3 s) vì trên một máy, shuffle chỉ là copy qua đĩa/loopback, gần như miễn phí, trong khi combiner thêm một process Python phải đọc, parse và sort lại 1 triệu bản ghi ở mỗi map. Trên cluster thật, shuffle đi qua mạng và reducer phải merge-sort hàng GB từ hàng trăm mapper; khi đó giảm 99.9 % lượng shuffle sẽ áp đảo chi phí trên và combiner thắng rõ.

Q6. Không. Trung bình của các trung bình là sai vì mỗi partition có số bản ghi khác nhau. Combiner phải emit cặp `(sum, count)` cục bộ và reducer mới tính `sum/count` (đã cài đặt ở phần stretch). Combiner phải có tính kết hợp và giao hoán, output cùng định dạng với input, vì Hadoop có thể chạy nó 0, 1 hoặc nhiều lần.

Q7. Mỗi mapper nạp file nhỏ `taxi_zone_lookup.csv` (265 dòng, phân phối bằng distributed cache `-files`) vào một dict và thay ID bằng tên zone, nên join không cần shuffle. Cách này hết hợp lý khi bảng "nhỏ" không còn vừa thoải mái trong bộ nhớ của từng task; khi đó phải dùng reduce-side join (shuffle cả hai tập theo key). Spark gọi ý tưởng này là **broadcast join** (L06).

Q8. `BAD_RECORDS` xuất hiện trong job counters (console cuối job và JobHistory UI → Counters → nhóm `TaxiLab`); chỉ xuất hiện khi có ít nhất một dòng hỏng. Vài dòng lỗi không nên làm chết cả job xử lý hàng tỷ dòng; đếm chúng giúp lộ ra vấn đề chất lượng dữ liệu và cho phép đặt ngưỡng cảnh báo.